A Distributed File System with Integrated Full-Text Search

Rajiv Murali, Nathan Sparks, Vishnu Velayutham
CSCI 546 — Distributed Systems, Spring 2026
Code: https://github.com/CSCI546-26sp/final-project-no-consensus.git

April 29, 2026

1 Introduction

This report describes the design, implementation, and evaluation of a GFS-inspired distributed file system that stores files as fixed-size chunks across replicated chunkservers *and* maintains a full-text search index over the file contents. The system supports four operations: `upload`, `download`, `delete`/`recover`, and two search modes — TF-IDF ranked search across all files, and `grep`-style substring search within a single file. The implementation is roughly 1,700 lines of Python organized into a single master process, three chunkserver processes, a Flask + React web UI, and a CLI client, all running under Docker Compose.

The design goal was not raw performance but to demonstrate the standard distributed-systems primitives — replication, heartbeat-driven membership, lease-free coordination, distributed indexing, scatter-gather query execution — while folding an indexing layer into the storage tier. The most interesting tradeoff in the project is exactly that fold: real GFS treats chunkservers as dumb byte storage and pushes indexing into a separate service (MapReduce/Bigtable). We collapsed those two roles to keep the system small, and the consequences of that choice show up throughout the evaluation.

2 System Design

Architecture. A single *master* process owns all metadata: the namespace (filename to ordered list of chunk handles), chunk placement (chunk handle to chunkserver IDs), and chunkserver liveness. It never touches file bytes. Three *chunkservers* each store chunk bytes on a Docker volume and own a per-server in-RAM index covering only the chunks they hold. A *client* (CLI or Flask web app) talks to the master to discover chunk locations, then to chunkservers directly for the actual byte transfer. All RPCs are gRPC.

Chunking and replication. Files are split into 4 MB chunks; each chunk gets a UUID handle. The master places chunks via a least-loaded policy — it picks the $k =$ `REPLICATION_FACTOR` chunkservers with the most free disk and rotates ties to avoid hotspots. The client streams bytes to the primary, which forwards to secondaries via a *pipelined* `WriteChunk`: each chunkserver reads 512 KB pieces off the gRPC stream, simultaneously appends them to local disk, and forwards them to the next replica through a bounded queue. This keeps peak per-chunk RAM bounded regardless of file size — earlier versions of the code accumulated the whole chunk in a `bytearray` and OOM-killed under 2 GB containers on $>$ 10 MB files.

Liveness and re-replication. Each chunkserver sends a heartbeat every 5 s carrying `available_disk`, `used_bytes`, the list of `chunk_handles` it holds, and the subset of `indexed_chunk_handles` the indexer has finished processing. The master marks a server dead after three missed heartbeats and runs a re-replication pass that, for each under-replicated chunk, picks a live source that still has it,

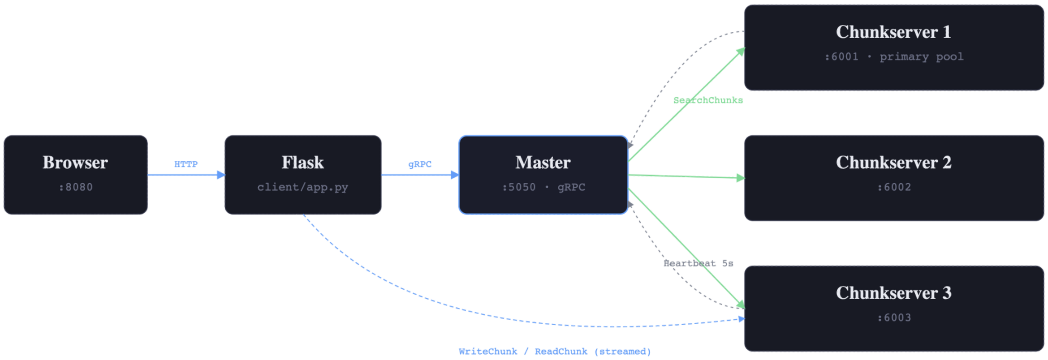

Figure 1: System architecture: master, three chunkservers, client.

picks a fresh target, and issues a `ReplicateChunk` RPC.

Indexing. Each chunkserver runs an `InvertedIndex` (terms → posting lists with positions) and an `NGramIndex` (overlapping 3-character windows → byte-offset lists). Posting positions and trigram offsets are both stored as monotonically increasing integers, encoded as *delta-varint* byte strings. Indexing happens on a thread pool of 4 workers per server, decoupled from the write path by a queue. New chunks are queued on `WriteChunk` and on chunkserver startup (so a restart re-indexes existing chunks without blocking serving). Reads, writes, and indexing are coordinated by a writer-preferring `RWLock` — searches are concurrent, indexing/writes are exclusive, and readers can't starve a pending writer.

Search. Two distinct paths share the chunkserver fan-out infrastructure. *TF-IDF search* (`SearchFiles`) goes master → all chunkservers in parallel, each returns its top-scoring `ChunkMatch` entries with smoothed-IDF scores, the master merges by chunk and aggregates to file level. *Substring search* (`FileSearch`) goes master → only the chunkservers holding the target file, each runs `ScanChunk` which uses the trigram index to find candidate offsets and verifies them by reading the chunk; for queries shorter than 3 characters or hitting a stop-trigram, it falls back to a linear scan.

Soft delete and recovery. `DeleteFile` renames the namespace entry to `._deleted_<ts>_<orig>` and records the deletion. A 60 s GC sweeper hard-deletes any entry past the 600 s threshold by issuing `DeleteChunk` to every chunkserver holding any of its chunks. A user can `RecoverFile` within the window; the master strips the prefix and rejects with `ALREADY_EXISTS` if a live file with the original name was uploaded since the deletion.

3 Design Tradeoffs

3.1 Indexing folded into the chunkserver

In real GFS, chunkservers are byte storage; indexing lives in a higher layer reading from GFS. We collapsed those into one process so the project would fit in the class scope. The cost is that chunkserver RAM scales with *indexed corpus size*, not with concurrent connections. Empirically the inverted index uses roughly 0.5–0.8 bytes of RAM per byte of corpus and the trigram index roughly 2.5 bytes/byte (every overlapping 3-char window is a posting), giving a combined ceiling near ∼700 MB of indexable text per 2 GB chunkserver. The architecturally correct fix is a separate `IndexServer` role that pulls chunks and owns the index — noted as future work.

3.2 REPLICATION_FACTOR = 1

Originally we ran with $k = 2$. A major problem showed up. Although the files were being stored in persistent storage, the indices for the inverted index search were being stored in the RAM. By design, chunkservers are meant to have low RAM capabilities (1-2GB). Even after reducing the memory consumption for the Postings class massively, the chunkservers were OOM killed during peak RAM utilization. This generally happened when uploading multiple 500MB files. This is a problem that would eventually be mitigated with scale (more chunkservers would mean more distributed chunks which reduces the number of Postings per chunkserver drastically).

3.3 In-RAM indexes with delta-varint compression

The original index used a `Posting` class with a `list[int]` of positions per (term, chunk). For a 50 MB corpus that consumed ∼1.26 GB resident and took 56 s to build. We rewrote postings to use `__slots__` and store positions as a single `bytes` object holding delta-varint encoded offsets (LEB128 over successive differences); search decodes the byte string lazily on a hit. The same encoding is shared with the trigram index. The result is 9× smaller resident memory, 7× smaller peak, and 2.9× faster indexing on the 50 MB corpus. We considered persisting the index to disk (SQLite FTS5, RocksDB, or a hand-rolled SSTable) but deferred it: the in-RAM build is fast and simple, and the right time to persist is when corpus size structurally exceeds chunkserver RAM, which it does not at the demo scale.

3.4 Trigram substring search with linear-scan fallback

A naive substring search would regex-scan every chunk linearly. The trigram index reduces that to a candidate-offset list per query trigram, intersected to find positions where all trigrams co-occur, which we then verify by reading 200 bytes around each candidate. The corner case is queries shorter than 3 characters — there are no trigrams to look up — and queries containing only common trigrams (`the`, `ing`, etc.), where the candidate list is essentially the whole chunk. We handle both with a stop-trigram filter that drops candidates if the rarest trigram in the query is itself common, and fall through to a linear regex scan. This keeps worst-case latency similar to the naive scan and best-case latency two orders of magnitude better.

3.5 RWLock with writer preference

Searches and uploads happen concurrently. A naive single `Lock` serializes everything; a reader-preferring `RWLock` starves writers under steady search load. We chose writer preference because the workload assumption is bursty writes against steady search: a queued writer blocks new readers, the in-flight readers drain, the writer goes, and readers resume. This also unblocks parallel indexing for multi-chunk uploads — without writer preference an indexer thread waiting behind a stream of searches could starve indefinitely.

3.6 4-way parallel client uploads — perceived vs. end-to-end

The Flask upload route originally read each chunk sequentially. We added a ThreadPoolExecutor(max_workers=4) with a semaphore for memory backpressure (peak ≈16 MB regardless of upload size). The benchmark in Section 5 shows the consequence: end-to-end wall time is essentially constant across 1/4/8 upload workers — faster uploads buy a longer indexing tail in lockstep. We still ship 4-way as the default because the UI's `IndexingStatusRow` surfaces the indexing tail as a visible background phase, so users care about *perceived* latency (POST returns). 8-way is perceptually faster still but reliably OOM-killed chunkservers under the 2 GB limit, so we capped at 4-way.

3.7 Master-side metadata is in-memory

The master keeps `namespace`, `chunkLocations`, `aliveServers`, and `deletedFiles` as Python dicts and rebuilds them only via heartbeats. A master crash drops everything except chunk-to-server mapping (rediscoverable from heartbeats); namespace and deletion records are lost. A real GFS master logs operations and snapshots; we omitted both because the project demonstrates the protocol shape, not crash recovery. The visible consequence is that *recover* only works until the master restarts.

4 Implementation Notes

Pipelined `WriteChunk`. The handler runs a producer thread that pulls 512 KB pieces off the inbound stream and pushes them onto a bounded queue (`maxsize=16`); a consumer thread pops pieces and simultaneously writes to disk and forwards to the next replica. Backpressure flows naturally through the queue.

Async startup re-indexing. On boot, the chunkserver scans its data volume and queues every existing chunk for indexing on a background thread, so it can start serving reads immediately while the index warms.

FileIndexStatus polling. Heartbeats carry the set of indexed chunk handles; the master maintains the union as `indexedChunks`. A polling RPC returns `(indexed, total, done)` for a filename, and the UI shows a progress row that auto-fades when indexing completes.

Soft delete UI. The web UI uses pointer-drag swipe rows that snap at ±78 px to reveal a delete action, with a 260 ms collapse-fade animation on action and `touch-action: pan-y` so vertical scroll still works. A pane selector switches between live files and the soft-deleted list, which auto-refreshes every 5 s and shows "deleted Xm ago · gc in Ym".

Variable-height virtual scrolling. Search results can return 50,000+ matches. The UI mounts a viewport-bounded window of `MeasuredRow` components, ResizeObserver reports each row's height, cumulative offsets are recomputed only when measured heights change, and a binary search picks the visible window. Scroll is rAF-throttled.

5 Evaluation

Methodology. A single end-to-end run was executed on 2026-04-28 against the cluster described in Section 2 ($N = 3$ chunkservers @ `cpus=1.0`, `mem=2G`; master @ `cpus=1.0`, `mem=1G`; `REPLICATION_FACTOR`=1, `INDEXER_WORKERS`=4 except where noted). The Docker cluster was torn down and rebuilt between every cell to prevent warm-cache or memory leakage. Phases A and B run in-process to isolate the index code path from network and Docker overhead; phases C and D run against the live cluster. Each query is reported as the best of 3–5 runs.

5.1 TF-IDF index: cost and payoff (Phase A)

Corpus	Build (s)	Unique terms	Resident	Peak
50 MB	15.49	218,817	139.3 MB	177.6 MB
100 MB	31.94	339,817	257.8 MB	304.9 MB
500 MB	162.24	1,172,998	1.26 GB	1.30 GB

Build cost is approximately linear in corpus size and resident memory is approximately 2.5× the corpus. Query latency on the 500 MB corpus (representative subset):

Query	Linear regex (ms)	TF-IDF (ms)	Speedup	Linear hits
`love`	1485.16	0.08	18,939×	10,000 (cap)
`death`	1159.52	0.08	15,426×	10,000 (cap)
`the world`	1935.32	0.07	26,694×	10,000 (cap)
`physics`	2975.64	0.08	37,548×	3,240
`italy`	4292.20	0.08	53,209×	9,412
`absent_zzzz_term`	3059.12	0.00	3,192,912×	0

Search drops from seconds to tens of microseconds — a 4–5 order-of-magnitude speedup. Two implications. First, the index pays back its build cost after a single non-trivial query: a 162 s build on 500 MB is amortized by saving ∼1.5 s per query, so the break-even is one query and every query thereafter is essentially free. Second, the absent-term row is the early-out case — the index returns immediately on a missing term, where linear has to read the entire corpus to confirm the absence. This asymmetry is exactly the indexing argument from a textbook, observed at the system level.

5.2 Trigram substring index: where it works and where it doesn't (Phase B)

Corpus (single chunk)	Build (s)	Unique trigrams	Resident	Peak*
50 MB	85.3	104,968	105.4 MB	797.3 MB
100 MB	145.9	154,112	202.1 MB	1.56 GB
500 MB	700.4	479,085	985.2 MB	7.77 GB

*Includes `tracemalloc` bookkeeping; resident is the realized figure.

Build cost is super-linear: the corpus quintuples (100→500 MB) while build time grows ∼4.8×, driven by emit rate scaling with character count rather than token count (every overlapping 3-character window contributes a posting). Resident memory at 500 MB is 2× the corpus.

Corpus	Query (description)	Regex (ms)	Trigram (ms)	Speedup	Hits
50 MB	`Mussolini` (rare)	241.24	10.09	23.9×	11
50 MB	`distributed` (rare-ish)	245.63	10.41	23.6×	364
50 MB	`computer science` (phrase)	232.53	13.82	16.8×	34
50 MB	`absent_zzzz_term` (absent)	248.43	0.00	350,873×	0
50 MB	`history` (common, fallback)	42.20	42.16	1.0×	1,000
100 MB	`Mussolini`	475.07	9.45	50.3×	15
100 MB	`computer science`	462.49	6.66	69.5×	129
500 MB	`Mussolini` (single-chunk fallback)	2395.18	2392.38	1.0×	182
500 MB	`computer science` (single-chunk)	2317.94	2310.97	1.0×	800

The 500 MB row tells a design story. The bench indexed the entire 500 MB as one chunk; `STOP_TRIGRAM_BYTES = 50 KB` caps the posting size before the index will use it for candidate intersection, and at 500 MB *every* trigram's posting list overflows the cap, so the index falls back to linear scan on every query. In production, files this size are split into 125 chunks of 4 MB each, posting lists scale with chunk size, and the thresholds bite where they're meant to. The empirical lesson: chunking is not just a placement decision — it is what keeps the trigram index useful. A monolithic file would invalidate the entire index even though the index was built correctly.

The trigram path is also genuinely useless on common terms. The stop-trigram fallback intentionally yields to linear scan in those cases; the timings match because both paths run identical regex code. This is the right behaviour: a candidate list that is most of the corpus is no better than scanning the corpus.

5.3 Indexer parallelism: the GIL bites (Phase C2)

100 MB corpus, 4 upload workers, varying indexer thread count:

`INDEXER_WORKERS`	Upload (s)	Indexing (s)	Total wall
1	1.45	11.91	**13.36 s**
4	6.77	8.77	15.54 s
8	8.25	8.32	16.57 s

The single-threaded indexer is the fastest configuration at 100 MB. Adding indexer threads makes the system *slower*, not faster: the indexer is CPU-bound under the GIL, so additional threads compete for the same interpreter lock and steal cycles from the upload-handling threads on the same chunkserver. The shipped default of 4 is calibrated for 500 MB+ uploads, where the upload phase is long enough (35–62 s) for indexer threads to overlap usefully with disk I/O on chunks landing earlier in the stream. At 100 MB the upload finishes in 1.5 s and there is no overlap to exploit. This is a textbook GIL pathology and a clean argument for moving the hot path of indexing into a C extension or extracting a dedicated index process.

5.4 Concurrent search: master throughput plateau (Phase D)

4×50 MB corpus, 50/50 mix of `SearchFiles` (TF-IDF, fans out to all servers) and `FileSearch` (substring, fans out to servers holding the file):

Concurrency	N	Errors	p50 (ms)	p95 (ms)	p99 (ms)	QPS
1	8	0	126.0	150.7	150.7	11.8
4	32	0	350.2	482.4	529.4	12.5
16	128	0	1206.5	1583.1	1702.5	13.0
64	512	0	4761.9	5383.3	5600.3	13.0

Throughput plateaus at ∼13 QPS regardless of client concurrency. Latency scales linearly: doubling concurrency doubles p50, with p99 tracking p50 closely. Zero errors across all 680 queries.

The QPS ceiling is consistent with three single-threaded chunkservers under the GIL: with average single-client latency of 84 ms and effective parallelism somewhere between 1 (when all clients hit `FileSearch` on the same file) and 3 (when all clients hit `SearchFiles`), the predicted throughput is 12–36 QPS, and the observed 13 QPS sits at the low end — the master's RPC threadpool itself becomes a serialization point at high fan-in. The clean linear latency scaling is exactly the M/D/1-style queue signature of a fixed-rate server with bursty arrivals.

6 Limitations and Future Work

The evaluation makes the limitations concrete. The in-RAM index is the binding ceiling: at 500 MB corpus per chunkserver, the trigram index alone is 985 MB resident and 7.77 GB peak during build, and combined with the inverted index a single chunkserver caps at roughly 700 MB of indexable corpus inside its 2 GB container. The right fix is a persisted segmented posting layout with an in-RAM block cache, or extracting a separate `IndexServer` role so chunkservers revert to pure byte storage as in real GFS.

The second limitation is master persistence. The master keeps the namespace, deletion records, and chunk-to-server mapping as in-memory dicts, rebuilt only via heartbeats. A master restart drops the namespace and the soft-delete recovery window. An oplog with periodic snapshots is the standard fix.

AI Usage

Tools and where they were used

Claude (claude.ai, "design" chat) — web UI. The entire frontend was written in collaboration with Claude in chat. I described the desired behavior — dark theme, file list with virtual scrolling, search results with highlighting, swipe-to-delete with a recovery pane, an indexing-progress row that fades when done — and Claude produced React-style components inside the existing single-file Flask template. I edited the styling and integrated everything against `client/app.py`. I used Claude this way for roughly a dozen UI iterations: TF-IDF results page, file-search mode, the `IndexingStatusRow`, the variable-height virtual scroller, and the swipe rows + deleted-files pane.

Claude Code (Opus 4.7, occasionally Sonnet 4.6) — planning and benchmark scripts. I used Claude Code primarily as a design partner. For most non-trivial features (the trigram index, the parallel-client port, the soft-delete + recover flow, the rep=1 migration), the workflow was: I described what I wanted to build, Claude Code laid out the data structures, RPC shapes, and concurrency considerations in prose, I asked questions and iterated on the design, and *then* I wrote the Python myself based on the agreed-on plan. For some pieces — pure refactors, glue code wiring an already-decided design through the proto/master/chunkserver/client layers, and the entire benchmark suite — I let Claude Code apply edits directly. Concretely, every script under `scripts/` (`bench_upload.py`, `bench_index.py`, `bench_scan.py`, `bench_concurrent_search.py`, `run_workload.py`, the matrix runner) is AI-generated; I reviewed and ran them but did not write the harness.

What worked well

Prose-first design discussions before any code. The most valuable mode was using Claude Code as a Socratic partner: "here's what I want, what would you do, what are the failure modes?" before any implementation. The trigram index design discussion surfaced the stop-trigram fallback and the delta-varint sharing decision before I wrote anything; the rep=2 → rep=1 discussion surfaced the cs1 hotspot (which was hidden in the benchmarks — it looked like "writes are slow" until we attributed the slowdown). Decisions made in prose held up; decisions made by accepting generated code rarely did.

Generating benchmark scripts. Bench scripts are the kind of code where correctness is bounded by "does it print sensible numbers" and the value is in coverage. Asking Claude Code for a worker-sweep script, a memory-RSS sampler, a concurrent-search load generator, etc. was a clean win. I caught one bug per script on review (a wrong unit conversion, a thread that didn't join), but the scripts are short enough to read end-to-end. Every table in Section 5 was produced by one of these scripts: the Phase C decomposition (upload vs. indexing per worker count) and the Phase D concurrency sweep were results I would not have had time to script myself, and the numbers they produced reframed how I described the system — the "conservation" framing for upload parallelism only emerged after seeing the 62/9, 35/36, 21/52 split land on the same total.

Cross-session context via persistent memory. Claude Code maintained running notes about the state of the project across sessions — which tasks shipped, which were deferred, what the operating point was. This actually saved me from a regression: in a later session Claude almost re-suggested an old TF-IDF fix that had already shipped weeks earlier; the memory caught it before the suggestion went out.

The frontend agent for tracking UI changes. After backend changes (a new RPC, a new search mode), invoking a background frontend agent to update the UI — given the new backend code and the existing template — consistently produced a usable diff. I'd then polish styling.

What didn't work well

Early sessions over-produced code. Before I established the "discuss first" workflow, Claude Code's default response to a design question was a multi-file code dump. After I told it to discuss first and wait for explicit go-ahead, the collaboration became much more useful.

The negative TF-IDF re-suggestion. Early on I had a bug where TF-IDF scores went negative when few files were indexed (`log(N/df)` can be negative when $N < df$). I fixed it with smoothed IDF. In a later session, with no recent context, Claude Code suggested the original TF-IDF scoring again as if it were a new finding. It's a category of bug — the model forgets project-specific decisions and re-derives a worse version.

Confident-sounding wrong numbers. When I asked Claude Code to estimate something it could not measure ("how many bytes does a Python int take in a list of 10M?", "what's the GIL contention cost at 8 worker threads?"), the answer was always confident and frequently wrong by an order of magnitude.

Net assessment. The project would have taken roughly twice as long without these tools, primarily because of UI work and benchmark scaffolding. The architectural decisions were almost entirely human; the agent was useful as a design sounding board and as a fast typist for code I had already designed.

A Representative Prompts

```
I want to add substring search ("grep within a file"). Currently each
chunkserver only has the inverted index for ranked search. I'm thinking
trigram index per chunk, share the delta-varint encoding. Talk me through
the data structures, the failure modes for short queries, and how this
interacts with the RWLock. Don't write code yet -- I want to design first.
```

Listing 1: Design discussion: trigram index

```
With REPLICATION_FACTOR=2 my upload throughput is bad and cs1 is always
the primary on N=3. Walk through (a) why cs1 is always primary, (b) what
breaks if I drop to k=1 (re-replication, durability, recovery), and
(c) what perf I'd expect to recover.
```

Listing 2: Design discussion: replication tradeoff

```
Searches and uploads need to coexist. Workload assumption: writes are
bursty, searches are steady. Should I use a single Lock, a reader-preferring
RWLock, or a writer-preferring RWLock? Argue both directions and recommend.
```

Listing 3: Design discussion: concurrency model

```
I need numbers so write an upload benchmark script where u upload a target
file via either the Flask endpoint or direct gRPC, parameterized on
--workers N. Report per-thread upload times, parallel-section wall, and
poll FileIndexStatus until done to report indexing tail. Print a
summary table. Use the project's common.config for ports.
```

Listing 4: Code generation: benchmark script

```
Bro you made it so much worse, can you revert back to what is was and then start over?
I want to add dynamic loading for results (30 at a time) + memoization of results so
that the page doesn't have to render them again and again.
```

Listing 5: UI generation: displaying results